

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

LEITURA E ESTRUTURAÇÃO DE DOCUMENTOS

Prof. Anderson França

MATRIZ CURRICULAR

Essa disciplina explora conceitos fundamentais da ciência de dados, incluindo técnicas de coleta, limpeza e análise de dados. Serão utilizados **algoritmos de aprendizado de máquina**, como regressão, árvores de decisão e redes neurais, aplicados à previsão e classificação. Além disso, serão utilizados métodos de automação de processos utilizando ferramentas modernas e técnicas avançadas para otimizar e escalonar soluções analíticas.



1. Introdução à análise e ciência de dados	44
2. Estatística aplicada I – Cultura orientada a dados e transformação digital	90
3. Estatística aplicada II – Ciência de dados, aprendizado de máquinas e otimização	136
4. <i>Deep learning</i> e inteligência artificial	50
5. Inteligência artificial aplicada	80

OBJETIVO DA AULA

O objetivo desta aula é capacitar os alunos a compreender e aplicar técnicas de leitura, extração e estruturação de informações a partir de documentos digitais, com foco em formatos amplamente utilizados em contextos públicos e regulatórios, como HTML e PDF.

Ao final da aula, os alunos serão capazes de:

- Entender as diferenças entre os principais formatos de documentos utilizados na web (HTML, PDF, CSV, DOCX) e suas implicações no processamento automatizado.
- Utilizar bibliotecas como BeautifulSoup para navegar em páginas HTML e extrair conteúdo específico (títulos, parágrafos, links).
- Realizar extração de texto em arquivos PDF utilizando ferramentas como pdfplumber e PyMuPDF, considerando a estrutura e as limitações desses formatos.
- Aplicar expressões regulares para identificar e extrair padrões úteis no texto, como e-mails, datas, números de processo e telefones.
- Organizar as informações extraídas em estruturas como listas, dicionários e DataFrames, preparando os dados para futuras etapas de análise ou modelagem.

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

TRANSFORMAR DOCUMENTOS EM DADOS



MINISTÉRIO DA
SAÚDE



TRANSFORMAR DOCUMENTOS EM DADOS

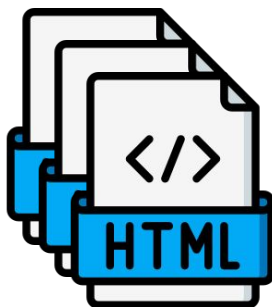


"Se eu te entregar um parecer técnico em PDF com 10 páginas, você conseguiria transformá-lo em uma base de dados analisável em 10 minutos?"

DOCUMENTOS DO DIA-A-DIA



PDF com muitas
páginas



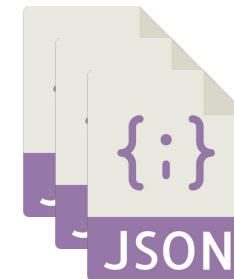
Parecer publicado
em páginas *web*



Documentos
criados no setor

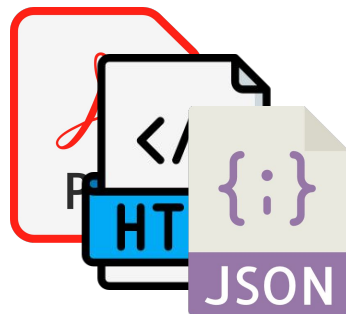


Logs de sistemas e
dados de auditoria



Respostas de apis e
sistemas internos

Todos são diferentes e
com o mesmo desafio:



Não são dados
AINDA

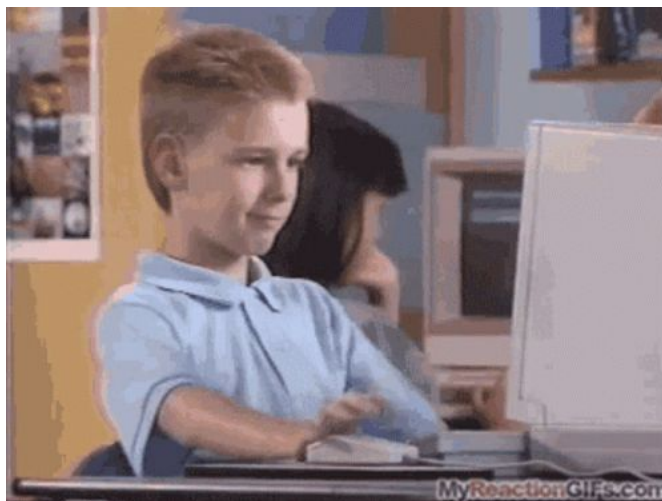
DOCUMENTOS DO DIA-A-DIA

O que encontramos nos documentos textuais são:

- Texto desorganizado, com formatação complexa
- Tabelas embutidas, notas de rodapé, múltiplas colunas
- Linguagem ambígua e estrutura variável

Todos possuem formatos diferentes, mas problema parecidos

Para humanos pode parecer uma tarefa simples



Para os algoritmos é uma tarefa extremamente complexa



DADOS ESTRUTURADOS vs TEXTO LIVRE

A maioria dos documentos contém informações importantes, mas em formato de texto livre. Isso é bem difícil de processar nos modelos de IA.

Para transformar esses conteúdos em dados utilizáveis, é preciso **identificar**, **extrair** e **organizar** os elementos relevantes em uma estrutura consistente e legível por máquina.

Esse processo é essencial para alimentar modelos estatísticos e de linguagem com qualidade e precisão.

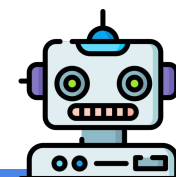
Tipo	Exemplo
Estruturado	{"substância": "Parabeno", "nível": 0.3}
Texto Livre	"Foi identificado o uso de parabenos em concentração de 0,3% no produto..."

O QUE PRECISAMOS EXTRAIR?



Modelos Estatísticos e Aprendizado de Máquinas

- Colunas
- Variáveis
- Datas
- Categorias



Modelos de Linguagem (LLMs)

- Texto Organizado
- Seções Claras
- Contexto Bem Definido

Lembrando:

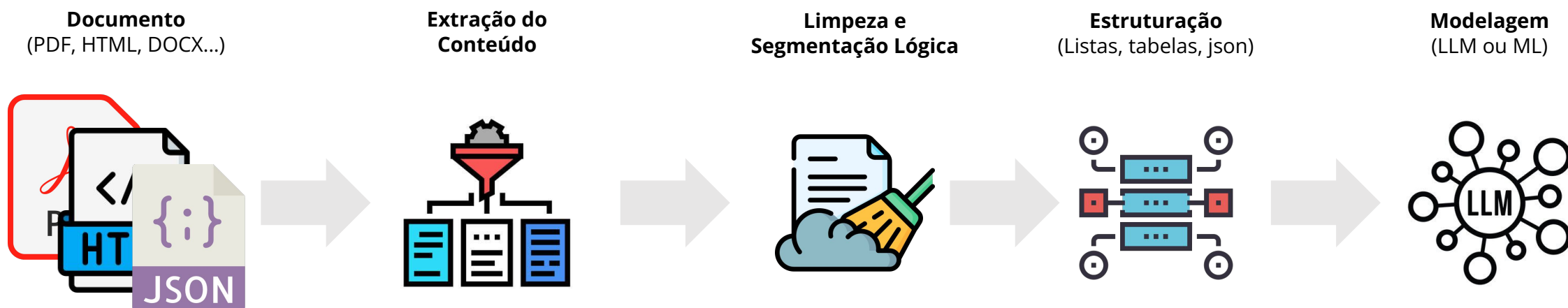
Garbage in, Garbage Out

Se a entrada é ruim, o resultado será ruim também

PIPELINE DE TRANSFORMAÇÃO

Modelos de IA e análises estatísticas exigem entradas organizadas. Documentos em PDF, HTML ou Word raramente vêm prontos para isso.

O processo de transformação envolve etapas bem definidas: extrair, limpar, estruturar e só então modelar.



INTERPRETAR, ESTRUTURAR E TRANSFORMAR

Ler documentos não é o mesmo que extrair dados

Entender a estrutura de um arquivo é o primeiro passo para qualquer análise automatizada. Cada documento carrega uma lógica própria, que é explícita para o leitor humano, mas invisível para algoritmos.

Transformar esse conteúdo em dados úteis exige conhecimento técnico, decisões contextuais e ferramentas adequadas.

A partir de agora, vamos explorar os diferentes formatos que vamos encontrar no dia a dia e os desafios específicos que cada um deles impõe ao processo de estruturação.

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

FORMATOS DE DADOS

DIFICULDADE GERAIS

Apesar de serem estruturas completamente diferentes, as principais dificuldades são:

- Separar conteúdo útil de metadados visuais (headers, rodapés, numeração)
- Manter contexto ao dividir por parágrafos ou seções
- Tratar quebras de linha, espaçamento e símbolos especiais
- Preservar o significado do conteúdo ao “quebrar” o documento em partes menores

Exemplo: Representações diferentes para um mesmo parecer técnico

 PDF	 HTML	 DOCX	 CSV
O parecer está em duas colunas, com logotipo institucional no cabeçalho, número do processo no rodapé e várias quebras de página. Visualmente bonito, mas difícil de extrair com precisão.	O mesmo parecer foi publicado em um portal oficial. O texto aparece em blocos divididos por <code><div></code> e <code><p></code> , e parte do conteúdo (como anexos) é carregado via JavaScript. Requer tratamento para eliminar menus, rodapés e scripts.	O documento original foi escrito no Word. Possui estilos aplicados nos títulos, sumário automático e tabelas organizadas. É o formato mais limpo para extrair, desde que não tenha sido convertido de forma inconsistente.	Um resumo do parecer foi exportado para CSV com colunas como: "Data", "Órgão", "Responsável", "Resumo". Ideal para consulta rápida ou cruzamento com outros dados, mas sem a riqueza de contexto do texto completo.

PDF (Portable Document Format)

É o formato mais comum quando falamos de documentos oficiais como pareceres, laudos e documentos regulatórios.

Ele foca em manter o visual do arquivo igual em qualquer lugar. O que é ótimo para leitura, mas complicado quando precisamos extrair e organizar os dados. Os principais formatos são: Nativos e escaneados.

Características:

- Pode conter texto real ou apenas imagem (escaneado)
- Estrutura visual não indica estrutura semântica
- Pode ter múltiplas colunas, tabelas ocultas ou elementos sobrepostos
- Difícil manter o layout original ao extrair

Pontos de Atenção:

- Nem todo PDF tem texto: se for escaneado, precisa de OCR
- Tabelas podem perder alinhamento na extração
- Cabeçalhos e rodapés se repetem e poluem o conteúdo



HTML (HyperText Markup Language)

É o que está por trás das páginas da internet, portais e sistemas online. Os textos são divididos por blocos e marcadores, o que ajuda na navegação, mas pode virar um desafio quando o conteúdo está espalhado ou misturado com elementos visuais.

Características:

- Estrutura marcada por tags (div, p, table)
- Conteúdo pode vir fragmentado por blocos
- Pode conter scripts e elementos interativos
- Facilidade em navegar com BeautifulSoup, mas requer filtragem

Pontos de Atenção:

- Pode conter ruído: menus, botões, scripts
- Texto pode vir fragmentado em várias tags
- Requer filtros para isolar o conteúdo principal
- Conteúdo pode estar em múltiplas páginas



Site Agência GOV



```
<div class="outer-wrapper">...</div>
<!--/outer-wrapper -->
<footer id="portal-footer-wrapper">...</footer>
...
<!-- Início VLibras Widget --> == $0
<!-- https://vlibras.gov.br/doc/widget/installation/webpageintegration.html
-->
<div vw class="enabled" style="left: initial; right: 0px; top: 50%; bottom:
initial; transform: translateY(calc(-50% - 10px));">...</div>
<script src="https://vlibras.gov.br/app/vlibras-plugin.js"></script>
<script> new window.VLibras.Widget('https://vlibras.gov.br/app'); </script>
<!-- Fim VLibras Widget -->
<footer>...</footer>
<script src="/++plone++ebc.agenciagov.stylesheets/script.js?v3"></script>
<script src="/++plone++ebc.agenciagov.stylesheets/limite.js?v3"></script>
<div id="plone-analytics">...</div>
<iframe allow="join-ad-interest-group" data-tagging-id="G-X0D00CT40G" data-
```

HTML do Site Agência GOV

DOCX

É o formato do Word, bastante usado para textos corridos, relatórios e documentos internos. Ele tem uma estrutura razoável (com títulos, tabelas, listas), o que facilita a leitura e também a extração, em muitos casos.

Características:

- Texto geralmente contínuo, com marcações de estilo (heading, bold, etc.)
- Tabelas e listas com estrutura clara
- Pode conter cabeçalhos e rodapés que se repetem
- Fácil de manipular com python-docx, mas limitado para layouts mais complexos

Pontos de Atenção:

- Pode perder estrutura se for salvo/exportado de forma incorreta
- Tabelas e listas nem sempre são lidas com precisão
- Cabeçalhos e rodapés precisam ser ignorados
- Podem sofrer alterações de estrutura entre uma entrega e outra



TXT/CSV

São formatos bem simples, usados quando se quer algo direto: texto puro ou tabelas separadas por vírgulas. São ótimos para dados mais limpos e controlados, mas limitados quando o conteúdo exige contexto ou formatação. Para textos simples, utilizamos muito o formato de texto (.txt), já para tabelas, é muito comum utilizarmos o valor separado por vírgula (.csv).

Observação importante: CSV NÃO é excel. Embora parecidos, o excel é uma planilha estruturada com abas, fórmulas, estilos, gráficos, etc. O .csv é um arquivo de texto simples.

Características:

- Simples, mas pouco informativo sobre estrutura
- Ideal para entradas muito controladas ou exportações bem definidas
- Não carrega hierarquia semântica (ex: seções, capítulos)

Pontos de Atenção:

- Fácil de quebrar: vírgulas dentro dos campos atrapalham
- Encoding mal definido causa erro na leitura (acentos, símbolos)
- Pode parecer organizado no Excel, mas vir desestruturado no código



COMPARANDO OS FORMATOS DE DOCUMENTOS

Cada formato possui um nível diferente de estrutura e complexidade para quem precisa transformar texto em dado.

- O PDF, apesar de ser o mais comum, é o mais desafiador, pois sua estrutura é visual, não semântica.
- HTML e DOCX já possuem marcações que ajudam a localizar e organizar o conteúdo, mas ainda exigem filtragem.
- Os CSV, por serem simples e estruturados, são os mais fáceis de trabalhar, desde que venham bem formatados.

Formato	Uso mais comum	Nível de estrutura	Dificuldade de extração
PDF	Documentos oficiais	Baixo (visual apenas)	Alta
HTML	Páginas da web	Médio (marcado por tags)	Média
DOCX	Relatórios internos	Médio (estilos aplicados)	Baixa
CSV	Exportações de sistemas	Alto (dados diretos)	Baixa

ENTENDER O FORMATO É O PRIMEIRO DESAFIO

Antes de qualquer extração, é essencial reconhecer o tipo de documento que estamos trabalhando.

Cada formato tem suas regras, limitações e armadilhas, e isso afeta diretamente a qualidade dos dados que vamos gerar.

Saber identificar e tratar essas diferenças é o que garante que, lá na frente, a análise ou modelo de IA esteja baseado em uma estrutura confiável.



O formato define estratégia.

Não existe extração automática sem entendimento prévio do problema.

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

EXTRAÇÃO DO CONTEÚDO BRUTO

EXTRAÇÃO DE DADOS

CIÊNCIA DE DADOS
E INTELIGÊNCIA ARTIFICIAL



Extração é o processo de acessar o conteúdo interno de documentos — texto, tabelas, seções — e transformar em algo que possamos manipular no código.

Aqui, o foco é **sair do "visual bonito" e chegar no "conteúdo utilizável"**.

ESTRATÉGIA DEPENDE DO FORMATO

Nem todo documento pode ser lido da mesma forma. A estratégia de extração depende do tipo de arquivo e de como o conteúdo está armazenado:

- texto real,
- imagem,
- marcações HTML ou
- estrutura de parágrafos.

Saber escolher a ferramenta certa é o primeiro passo para acessar a informação com precisão.

Formato	Ferramenta Indicada	Tipo de Conteúdo
PDF	pdfplumber, PyMuPDF	Pode ser texto ou imagem
HTML	BeautifulSoup, lxml	Texto com marcação por tags
DOCX	python-docx	Texto estruturado
Escaneado	Tesseract, EasyOCR	Imagem (precisa de OCR)

PERGUNTA ESTRATÉGICA 1

Se hoje temos muitas ferramentas como LLM, por que ainda precisamos trabalhar as técnicas clássicas?

1. **O documento tem layout previsível e recorrente**
Ex: pareceres da Anvisa, laudos técnicos, formulários padronizados
2. **Você precisa de alta precisão e controle sobre os dados extraídos**
Ex: identificar com 100% de certeza o CNPJ, data de emissão, substâncias etc.
3. **O volume é grande, mas o conteúdo é repetitivo ou bem estruturado**
4. **O custo computacional ou a privacidade impedem o uso de LLMs na nuvem**
5. **Você quer alimentar um LLM com dados já limpos** (e não deixar o LLM “entender tudo sozinho”)

Ferramentas como [pdfplumber](#), [pymupdf](#), [BeautifulSoup](#), regex e afins ainda são **insubstituíveis quando o objetivo é padronização, integração com bancos de dados, ou automação confiável e validável.**

PERGUNTA ESTRATÉGICA 2

Então quando faz mais sentido usar LLM direto (ou via RAG)?

Modelos como GPT-4, DeepSeek, Claude, Mistral etc. começam a ser mais vantajosos quando:

1. **O layout do documento é altamente variável ou desestruturado**
Ex: contratos, atas, documentos jurídicos, pareceres extensos com linguagem natural
2. **Você quer extrair insights sem necessariamente estruturar tudo**
Ex: "Esse parecer autoriza o uso dessa substância?", "Quem é o responsável técnico?"
3. **Você precisa responder perguntas sobre o conteúdo, e não apenas extrair campos**
4. **Você tem múltiplos formatos ao mesmo tempo**
HTML + PDF + DOCX + e-mails
5. **Você deseja classificar, resumir ou tomar decisões contextuais a partir do texto**

A NOVA GERAÇÃO DE LEITURA DOCUMENTAL

CIÊNCIA DE DADOS
E INTELIGÊNCIA ARTIFICIAL

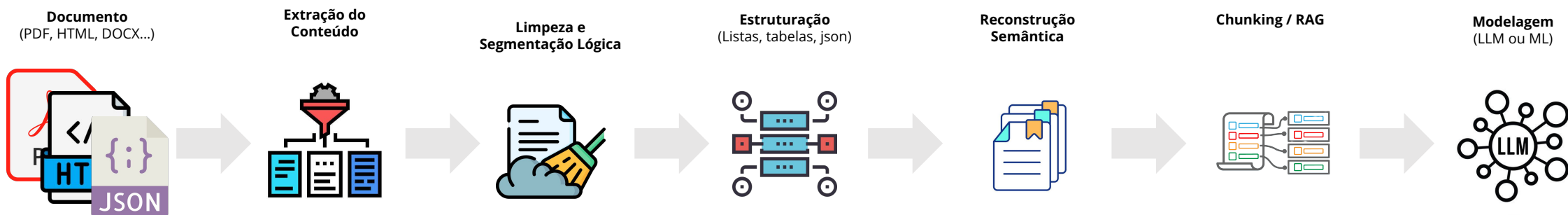
Antigamente*

- OCR
- regex
- texto linear
- parsing manual

Atualmente

- parsing multimodal
- reconstrução estrutural
- markdown semântico
- chunking inteligente
- preparação para LLM/RAG

*Nem tão antigamente assim



Com a evolução tecnológica, o desafio moderno deixou de ser apenas “extrair texto”.
Agora precisamos preservar estrutura, contexto e significado.

DA ESTRUTURA À ANOTAÇÃO: DANDO SIGNIFICADO AO TEXTO

Até aqui, extraímos informações brutas do documento e organizamos em campos como número de processo, substâncias, datas e assinaturas.

Agora, damos um passo além: vamos **anotar semanticamente** o conteúdo para deixar explícito **o que cada informação representa e qual o seu papel na decisão final**.

Isso vai muito além de regras fixas ou expressões regulares. A anotação semântica nos permite **descrever o significado do texto para a máquina**, transformando conteúdo livre em **conhecimento estruturado e reutilizável**.

Texto Original

A substância monoetilenoglicol (CAS 107-21-1) foi aprovada para uso em coloração capilar em 27/09/2024.

Apesar disso, o parecer destaca **risco de toxicidade** em certos contextos de uso.

Anotação

Tipo de Anotação	Trecho Marcado
Substância	monoetilenoglicol
Número CAS	107-21-1
Data Assinatura	27/09/2024
Conclusão Favorável	foi aprovada para uso em coloração capilar
Alerta de Risco	risco de toxicidade

Por que isso importa?

Modelos de linguagem (LLMs) não entendem estrutura por padrão.

A anotação semântica funciona como um "mapa" que guia o modelo sobre o que cada parte do documento significa.

TIPOS DE ANOTAÇÕES

A anotação semântica transforma texto em conhecimento estruturado. Mas cada tipo de anotação tem um propósito específico — ela nos ajuda a extrair, interpretar e aplicar o conteúdo de formas diferentes:

Entidades Nomeadas (NER)	Classificação de trechos	Blocos estruturais do documento	Relações entre entidades
São os “objetos” do texto: coisas, pessoas, números técnicos.	Define o significado funcional de uma sentença ou parágrafo.	Identifica seções do documento.	Liga elementos entre si para gerar contexto.
• Substâncias químicas • Números CAS • Datas • Órgãos reguladores • Responsáveis técnicos	• Conclusão favorável • Alerta de risco • Proposta de uso • Fundamentação técnica	• Relatório • Análise • Conclusão • Referências	• Substância X: uso pretendido Y • Responsável X: assinou o parecer em data Y • Substância X: classificada como risco Y

Essas anotações podem ser feitas manualmente, com auxílio de ferramentas, ou extraídas com modelos de IA. O importante é **tornar o texto legível para a máquina com significado explícito.**

FERRAMENTAS PARA ANOTAÇÃO

A anotação pode ser feita de forma manual, semiautomática ou com apoio de modelos. Essas ferramentas ajudam a marcar o texto, visualizar categorias e exportar os dados de forma estruturada.



Docling

- Foco em documentos estruturados e técnicos (PDFs, pareceres, regulatórios)
- Permite marcar entidades e definir estrutura semântica
- Pensado para legibilidade e rastreabilidade por máquinas e humanos
- Exporta em JSON estruturado, pronto para validação ou RAG
- Excelente para áreas como saúde pública, jurídica, regulatória



- Foco em NLP supervisionado (NER, classificação, sequência)
- Interface simples, rápida e funcional
- Ótimo para equipes pequenas e protótipos de anotação textual
- Muito usado para montar datasets de NLP

 **Label Studio**



- Open source, altamente configurável
- Suporta anotação de texto, imagem, áudio, vídeo
- Ideal para projetos colaborativos e multimodais
- Muito usado em empresas e laboratórios de IA
- Mais genérico e flexível

RegEx
Regular Expression

- Ótimos para anotações automatizadas em massa
- Úteis como etapa de pré-anotação antes da revisão humana
- Altamente integráveis com Python e pandas

A escolha da ferramenta depende do tipo de anotação, volume de documentos e nível de automação que queremos trabalhar. Muitas vezes começamos com regex, revisamos manualmente no Docling ou Doccano, e depois usamos esse material para treinar ou validar modelos.



CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

HTML BÁSICO PARA WEB SCRAPING

HTML (HyperText Markup Language)

É o que está por trás das páginas da internet, portais e sistemas online. Os textos são divididos por blocos e marcadores, o que ajuda na navegação, mas pode virar um desafio quando o conteúdo está espalhado ou misturado com elementos visuais.

Características:

- Estrutura marcada por tags (div, p, table)
- Conteúdo pode vir fragmentado por blocos
- Pode conter scripts e elementos interativos
- Facilidade em navegar com BeautifulSoup, mas requer filtragem

Pontos de Atenção:

- Pode conter ruído: menus, botões, scripts
- Texto pode vir fragmentado em várias tags
- Requer filtros para isolar o conteúdo principal
- Conteúdo pode estar em múltiplas páginas



Site Agência GOV



```
<div class="outer-wrapper">...</div>
<!--/outer-wrapper -->
<footer id="portal-footer-wrapper">...</footer>
...
<!-- Início VLibras Widget --> == $0
<!-- https://vlibras.gov.br/doc/widget/installation/webpageintegration.html
-->
<div vw class="enabled" style="left: initial; right: 0px; top: 50%; bottom:
initial; transform: translateY(calc(-50% - 10px));">...</div>
<script src="https://vlibras.gov.br/app/vlibras-plugin.js"></script>
<script> new window.VLibras.Widget('https://vlibras.gov.br/app'); </script>
<!-- Fim VLibras Widget -->
<footer>...</footer>
<script src="/++plone++ebc.agenciagov.stylesheets/script.js?v3"></script>
<script src="/++plone++ebc.agenciagov.stylesheets/limite.js?v3"></script>
<div id="plone-analytics">...</div>
<iframe allow="join-ad-interest-group" data-tagging-id="G-X0D00CT40G" data-
```

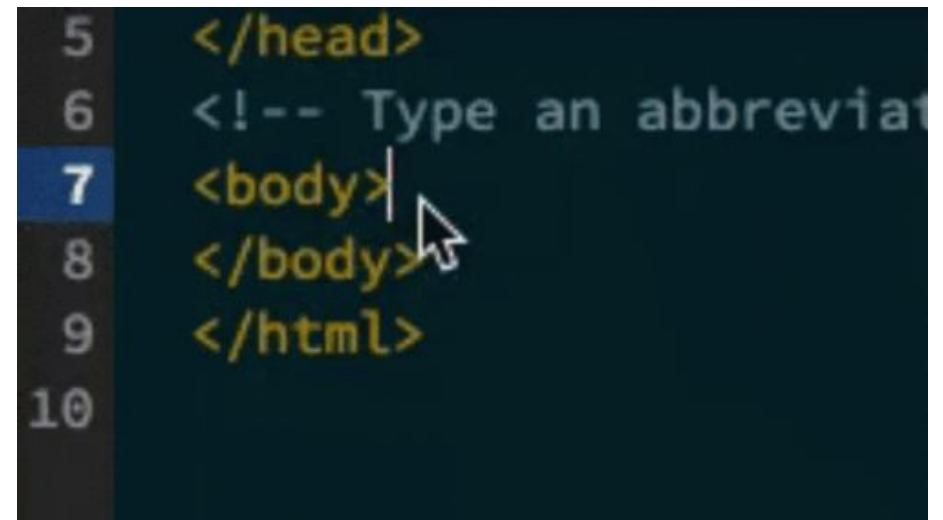
HTML do Site Agência GOV

LEITURA DE HTML (SIMPLES)

Muitos documentos públicos, como pareceres técnicos, relatórios ou atas, são publicados diretamente em portais governamentais ou sistemas internos via HTML. Isso nos permite acessar o conteúdo de forma mais leve, mas exige filtragem.

Por que começamos com HTML?

- **É acessível:** basta uma URL ou o código da página
- **Tem estrutura marcada:** com tags como `<p>`, `<div>`, `<table>`, etc.
- **Requer limpeza:** menus, scripts e rodapés poluem o conteúdo
- **É ótimo para treinar leitura semântica de blocos**



```
5 </head>
6 <!-- Type an abbreviat
7 <body>
8 </body>
9 </html>
10
```

ESTRUTURA BÁSICA DE UMA PÁGINA

Toda página web começa com a estrutura ao lado. Os elementos principais são:

- **<head>**: informações sobre a página (título, configurações, css, javascript)
- **<body>**: onde fica o conteúdo visível e que geralmente estão os dados que queremos (títulos, parágrafos, links, imagens)

Essa estrutura organiza a página em blocos que podem ser acessados pelas nossas ferramentas de scraping.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Minha Página</title>
  </head>
  <body>
    <h1>Título</h1>
    <p>Um parágrafo.</p>
    <a href="https://site.com">Link</a>
  </body>
</html>
```


PRINCIPAIS ELEMENTOS QUE USAMOS EM SCRAPING

As páginas web são construídas com **HTML**, uma linguagem que organiza o conteúdo por meio de **tags**.

As **tags** funcionam como “etiquetas” que indicam o tipo de informação que está sendo exibida, como texto, imagem, link ou tabela. Elas sempre aparecem entre sinais de menor e maior, como <p> ou <div>, e quase sempre têm uma abertura e um fechamento:

Por exemplo:

```
<p>Este é um parágrafo</p>
```

As tags mais comuns em tarefas de *web scraping* são:

Tag	Função	Descrição
<p>	Parágrafo	Define um bloco de texto corrido, como em textos e descrições.
<a>	Link	Cria um link clicável. O endereço está no atributo href .
<div>	Bloco de conteúdo	Agrupa elementos em blocos. Muito usado para organizar o layout.
<h1>, <h2>...	Título e subtítulo	Representam títulos hierárquicos. <h1> é o mais importante.
	Imagem	Insere uma imagem na página. A fonte do arquivo está no atributo src.
<table>	Tabela	Organiza dados em linhas e colunas. Ideal para informações tabulares.

TAGS ESTRUTURAIS

Nem todo elemento HTML representa texto ou imagem. Algumas tags servem para organizar e agrupar conteúdo. Portanto, quando buscamos uma tag como `<p>`, `<span>` e demais tags, elas vão estar dentro de outros blocos. Essas estruturas ajudam a organizar o layout da página e são muito importantes para tarefas de *scraping*. As principais são:

Tag	Função	Descrição
<code><div></code>	Bloco de Conteúdo	Agrupa elementos em seções. Muito usada para estruturar o layout de páginas.
<code></code>	Destaque em Linha	Agrupa trechos curtos de texto, sem quebrar a linha. Útil para aplicar estilo ou identificar partes específicas.
<code><section></code>	Seção temática	Agrupa conteúdo relacionado. Tem significado semântico (ex: uma sessão de artigos).
<code><article></code>	Conteúdo Independente	Representa um conteúdo completo e autônomo, como uma notícia ou post.
<code><header></code>	Cabeçalho	Define a parte superior de uma seção ou página (título, menu, logo...). Geralmente igual para todas as páginas do mesmo site
<code><footer></code>	Rodapé	Define a parte inferior de uma seção ou página (dados de contato, links úteis).

ATRIBUTOS HTML

As tags HTML podem conter atributos, que trazem informações adicionais sobre o conteúdo.

- Um atributo aparece dentro da tag de abertura.
- Ele sempre segue o formato: **nome="valor"**
- Os atributos ajudam a identificar, descrever ou configurar os elementos da página.

Exemplo:

```
<a href="https://site.com" class="botao">Clique aqui</a>
```

Neste caso:

- **href**: define o destino do link
- **class**: agrupa o elemento em uma categoria para aplicar estilo ou localizar no scraping

Atributo	Descrição
href	Caminho de um link (usado na tag <a>)
src	Caminho de uma imagem (usado no)
alt	Texto alternativo da imagem

Atributo	Descrição
class	Agrupamento de elementos com a mesma função visual/estrutural
id	Identificador único na página
name	Usado em formulários ou campos de entrada

HTML COM ATRIBUTOS

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo de Página</title>
  </head>
  <body>
    <!-- Título principal da página -->
    <h1 id="titulo-principal">Bem-vindo ao site da Anvisa</h1>

    <!-- Parágrafo com uma classe -->
    <p class="descricao">Aqui você encontra informações atualizadas sobre saúde pública.</p>

    <!-- Link com atributos -->
    <a href="https://www.gov.br/anvisa" class="botao-link">Acesse o site oficial</a>

    <!-- Imagem com caminho e texto alternativo -->
    

    <!-- Tabela com dados fictícios -->
    <table class="dados">
      <tr>
        <th>Data</th>
        <th>Assunto</th>
      </tr>
      <tr>
        <td>2025-05-06</td>
        <td>Nova regulamentação de medicamentos</td>
      </tr>
    </table>
  </body>
</html>
```

- **id="titulo-principal"** - identifica de forma única o título na página
- **class="descricao"** e **class="botao-link"** - agrupam elementos para estilização ou seleção via scraping
- **href="..."** - define o destino do link
- **src="..."** e **alt="..."** - definem a imagem e o texto alternativo

HIERARQUIA SIMPLES

```
<body>
  <header>
    <h1>Logotipo da Anvisa</h1>
    <nav>
      <a href="/noticias">Notícias</a>
      <a href="/contato">Contato</a>
    </nav>
  </header>

  <main>
    <section class="lista-noticias">
      <article class="noticia">
        <h2 class="titulo-noticia">Anvisa publica nova regulamentação</h2>
        <p class="data">Publicado em: 06/05/2025</p>
        <div class="conteudo">
          <p>A nova norma trata da venda de canetas emagrecedoras...</p>
          <a href="/noticia-completa">Leia mais</a>
        </div>
      </article>
    </section>
  </main>

  <footer>
    <p>Agência Nacional de Vigilância Sanitária</p>
  </footer>
</body>
```

Explicação da hierarquia:

- **<body>** contém todo o conteúdo visível.
- Dentro dele, temos três grandes blocos:
 - **<header>** com o menu e logotipo
 - **<main>** com a seção principal de notícias
 - **<footer>** com informações finais
- As notícias estão dentro de **<article>** (bloco semântico), agrupadas por uma **<section>**.
- Dentro do artigo:
 - Título (**<h2>**)
 - Data (**<p>**)
 - Texto da notícia em uma **<div>**
 - Link para mais conteúdo (**<a>**)

BEAUTIFULSOUP

O BeautifulSoup é uma biblioteca Python usada para extrair e navegar em **dados de páginas HTML e XML**. Ele é leve, simples e extremamente útil para acessar e organizar informações que estão estruturadas na web.

BeautifulSoup

É uma biblioteca muito útil, pois:

- Permite ler o conteúdo de **qualquer página HTML estática**
- Acessa facilmente tags como `<p>`, `<div>`, `<table>`
- Filtra elementos por classe, id, nome ou estrutura
- Ideal para coletar dados públicos, organizar textos e automatizar consultas

BEAUTIFULSOUP

Vamos usar a biblioteca **BeautifulSoup** para acessar e extrair o conteúdo textual de uma notícia publicada no site da Anvisa.

O código realiza três etapas:

1. **Faz uma requisição HTTP** para obter o conteúdo HTML da página usando **requests**.
2. **Interpreta o HTML** com o **BeautifulSoup**, permitindo navegar pela estrutura do documento.
3. **Extrai todos os parágrafos (<p>)** da página, limpa os espaços e imprime somente os trechos com texto real.

```
from bs4 import BeautifulSoup
import requests

url = "https://www.gov.br/anvisa/pt-br/assuntos/noticias-anvisa/2025/confira-os-destaques-da-6a-reuniao-publica-da-dicol-de-2025"

resposta = requests.get(url)

soup = BeautifulSoup(resposta.text, "html.parser")
conteudo = soup.find_all("p")

for paragrafo in conteudo:
    texto = paragrafo.get_text(strip=True)
    if texto:
        print(texto)
```

INSPECIONAR ELEMENTOS

Para fazer scraping com precisão, precisamos encontrar exatamente onde está o dado na página. Para isso, usamos a ferramenta Inspeccionar Elemento do navegador. Para inspecionar o código da página, basta apertar **F12** ou clicar com o **botão direito no elemento da página** e em seguida em **inspecionar elemento**.

Exemplo: [Resolução RDC N° 551, de 30 de agosto de 2021](#)

The image shows a screenshot of the Diário Oficial da União website. The page header includes links for 'VERSÃO CERTIFICADA', 'DIÁRIO COMPLETO', and 'IMPRESSÃO'. The main content area displays the official seal and the title 'DIÁRIO OFICIAL DA UNIÃO'. Below this, it indicates the publication date 'Publicado em: 31/08/2021 | Edição: 165 | Seção: 1 | Página: 139' and the issuing body 'Órgão: Ministério da Saúde/Agência Nacional de Vigilância Sanitária/Diretoria Colegiada'. The main heading of the document is 'RESOLUÇÃO RDC N° 551, DE 30 DE AGOSTO DE 2021'. A snippet of the text is visible: 'Dispõe sobre a obrigatoriedade de execução e notificação de ações de campo por detentores de registro de produtos para a saúde no Brasil.'

On the right side, the browser's developer tools are open, displaying the 'Elements' panel. The HTML structure is shown, with the following code visible:

```
<div class="clearfix journal-content-article" data-analytics-asset-id="3416/2891" data-analytics-asset-title="RESOLUÇÃO RDC N° 551, DE 30 DE Agosto DE 2021" data-analytics-asset-type="web-content">
  <div class="dou-modelo">
    <div class="portlet-decorate left-underline"></div>
    <div class="social-media-share border-top border-bottom py-2"></div>
    <ul class="barra-botoes-materia text-center"></ul>
    <article id="materia">
      <div class="row-fluid">
        <div class="cabecalho-dou text-center">
          <div class="brasao-brasil" alt="Cabeçalho - Brasão do Brasil" title="Cabeçalho - Brasão do Brasil"></div>
          <h2 class="cabecalho-titulo-dou">Diário Oficial da União</h2>
        </div>
        <div class="detalhes-dou"></div>
        <div class="texto-dou">
          <p class="identifica">RESOLUÇÃO RDC N° 551, DE 30 DE Agosto DE 2021</p>
          <p class="ementa"></p>
          <p class="dou-paragraph"></p>
          <p class="dou-paragraph"></p>
          <p class="dou-paragraph"></p>
          <p class="dou-paragraph"></p>
          <p class="dou-paragraph">Art. 3º Para efeito desta Resolução são adotadas as seguintes definições:</p>
        </div>
      </div>
    </article>
  </div>
</div>
```


CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

EXERCÍCIOS EXTRAÇÃO REQUESTS

PROBLEMA 1 - CREATINAS

Imagine que surgiu a necessidade de extrair o conteúdo de uma notícia sobre resultados de análise em suplementos publicada no portal da Anvisa para fins de documentação, análise ou arquivamento estruturado.

Nosso objetivo é acessar a seguinte página diretamente pelo Python:

<https://www.gov.br/anvisa/pt-br/assuntos/noticias-anvisa/2025/creatinas-anvisa-divulga-resultados-de-analise-em-suplementos>

A partir dessa URL, vamos desenvolver um código que:

- Acesse a página HTML e identifique apenas os parágrafos relevantes do conteúdo principal
- Limpe o texto extraído, removendo espaços desnecessários ou parágrafos em branco
- Armazene o conteúdo de duas formas:
 - Como uma lista de parágrafos individuais
 - Como um único texto corrido, separado por quebras de parágrafo
- Extrair a tabela, se houver

Creatinas: Anvisa divulga resultados de análise em suplementos

Resultados de laboratório apontaram diversas incorreções de rotulagem, mas teores dos produtos estavam dentro do esperado.



Agência Gov | Via Anvisa

23/04/2025 16:32



PROBLEMA 2 - ANTAGONISTAS GLP-1

Temos que estruturar o conteúdo de uma notícia publicada no portal da Anvisa, relacionada à regulamentação de medicamentos utilizados para emagrecimento.

Essa notícia será utilizada em um relatório interno e precisa estar disponível em formato limpo e estruturado, contendo apenas os textos principais da matéria.

Seu objetivo é acessar a seguinte página diretamente com Python:

<https://www.gov.br/anvisa/pt-br/assuntos/noticias-anvisa/2025/canetas-em-agregadoras-so-poderao-ser-vendidas-com-retencao-de-receita>

A partir dessa URL, vamos desenvolver um código que:

- Acesse a página HTML e identifique apenas os parágrafos relevantes do conteúdo principal
- Limpe o texto extraído, removendo espaços desnecessários ou parágrafos em branco
- Armazene o conteúdo de duas formas:
 - Como uma lista de parágrafos individuais
 - Como um único texto corrido, separado por quebras de parágrafo
- Extrair a tabela, se houver

Anvisa obriga retenção de receita para venda de canetas como Ozempic

Para agência, retenção é necessária para aumentar o controle do uso

ANDRÉ RICHTER - REPÓRTER DA AGÊNCIA BRASIL

Publicado em 16/04/2025 - 17:40
Brasília



© REUTERS/HOLLIE ADAMS/PROIBIDA REPRODUÇÃO

PROBLEMA 3 - DOU

Você foi designado para automatizar a extração de publicações recentes do Diário Oficial da União (DOU). O objetivo é incorporar essas informações em sistemas internos de análise e acompanhamento de normativas governamentais.

A partir da seguinte URL:

<https://www.in.gov.br/web/dou/-/portaria-gm/ms-n-6.882-de-22-de-abril-de-2025-627636172>

Desenvolva um código em Python que:

- Acesse o conteúdo da página de leitura do DOU usando **requests**
- Identifique e extraia os **títulos das publicações** exibidas no dia atual
- Extraia, se possível, os **resumos ou trechos principais** associados a cada título
- Armazene essas informações de forma estruturada, como uma **lista de dicionários** contendo:
 - "titulo"
 - "resumo"



DIÁRIO OFICIAL DA UNIÃO

Publicado em: 06/05/2025 | Edição: 83 | Seção: 1 | Página: 113

Órgão: Ministério da Saúde/Agência Nacional de Saúde Suplementar/Diretoria Colegiada

DECISÃO DE 31 DE MARÇO DE 2025

A Diretoria Colegiada da AGÊNCIA NACIONAL DE SAÚDE SUPLEMENTAR - ANS, no uso de suas atribuições legais, e tendo em vista o disposto no inciso VI do artigo 10 da Lei nº 9.961, de 28 de janeiro de 2000 em deliberação através da 618ª Reunião de Diretoria Colegiada - DC Ordinária, realizada em 10 de fevereiro de 2025, julgou o seguinte processo administrativo:

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

TRABALHANDO COM PDF



MINISTÉRIO DA
SAÚDE



LEITURA DE PDF

Para esse exemplo, vamos selecionar um [parecer técnico nº 13, de 2024](#) – Definição da segurança de uso da substância Monoetilenoglicol. (selecionado de forma aleatória)

Características do Documento:

- Documento oficial da Anvisa em formato PDF.
- Contém seções como introdução, fundamentação técnica e conclusão.
- Possui formatação padronizada, facilitando a extração de texto.

Informações que serão extraídas:

- Número do Processo
- Substâncias Citadas
- Seção da Conclusão
- Responsável Técnico



PARECER Nº 13/2024/SEI/GGCOS/DIRE3/ANVISA

Processo nº 25351.903963/2022-21
Interessado: Gerência-Geral de Cosméticos e Saneantes (GGCOS)
Assunto: Avaliação de Segurança da Substância Monoetilenoglicol -
Nomenclatura Internacional de Ingrediente Cosmético (INCI):
Glycol (CAS 107-21-1)

Definição da segurança de uso da substância Monoetilenoglicol - INCI: Glycol (CAS 107-21-1) em produtos de higiene pessoal, cosméticos e perfumes, especialmente em produtos que entram em contato com a mucosa bucal

1. Relatório

A Gerência de Hemo e Biovigilância e Vigilância Pós-Uso de Alimentos, Cosméticos e Produtos Saneantes (GHBIO) da Anvisa foi comunicada em 05/09/2022 pelo Ministério da Agricultura, Pecuária e Abastecimento - MAPA, de investigação envolvendo intoxicação e óbito de animais de companhia após ingestão de petiscos próprios para alimentação animal que apresentaram contaminação do ingrediente Propilenoglicol com Etilenoglicol.

A Gerência de Inspeção e Fiscalização Sanitária de Alimentos, Saneantes e Cosméticos da Anvisa abriu dossiê de investigação, inicialmente para apurar se há risco para a saúde humana e, caso necessário, tomar as medidas sanitárias cabíveis; visto ser imprescindível a verificação da rastreabilidade dos insumos contaminados, para avaliar se o produto foi utilizado por empresa do setor de alimentos de uso humano.

PDFPLUMBER

O pdfplumber é uma ferramenta leve, prática e eficiente para trabalhar com PDFs que contêm texto real (ou seja, não escaneados).

Ele permite acessar o conteúdo por página, mantendo a ordem dos parágrafos, o que é ideal para documentos como pareceres técnicos, que seguem um formato textual tradicional. Vamos começar por essa biblioteca, pois:

- É fácil de instalar e usar com poucas linhas de código
- Suporta extração de **tabelas**, caso o documento tenha
- Permite acessar a **posição do texto** se necessário (embora esse recurso seja mais avançado)

```
pip install pdfplumber  
import pdfplumber
```

EXTRAIR TEXTOS DE UM PDF

Agora que já temos a biblioteca carregada, o próximo passo é **abrir o arquivo PDF** que queremos analisar.

Usamos a estrutura **with** para garantir que o arquivo seja aberto corretamente e fechado automaticamente ao final.

Dentro desse bloco, podemos acessar as páginas do documento e extrair o texto de forma segura e organizada.

```
# Carregar apenas uma página do documento
with pdfplumber.open(caminho) as pdf:
    texto_pag1 = pdf.pages[0].extract_text()
    print(texto_pag1)
```



o `extract_text()` retorna o conteúdo da página como uma string contínua.

```
# Carregar todas as páginas do documento
texto_completo = ""
with pdfplumber.open(caminho) as pdf:
    for pagina in pdf.pages:
        texto_completo += pagina.extract_text(layout=True) + "\n"

print(texto_completo[:1000]) # visualizar primeiros 1000 caracteres
```



Quando usamos esse laço para juntar o conteúdo de todas as páginas, estamos transformando um documento visual em uma **única string contínua de texto**.

O QUE MAIS PODEMOS FAZER

Além de extrair texto, o **pdfplumber** permite acessar **informações estruturais e detalhes do layout do PDF**.

Com ele, podemos:

- Ler **todas as páginas** do documento
- Identificar **quantas páginas** o PDF tem
- Acessar a **posição (x, y)** de cada trecho de texto
- Extrair **tabelas organizadas** com **extract_table()**

Cada página do PDF se comporta como um objeto. Podemos acessar esses objetos através das funções:

```
pagina.width          # largura da página
pagina.height         # altura da página
pagina.chars          # lista com cada caractere, posição e fonte
pagina.extract_text()  # texto da página
pagina.extract_table() # tabela, se houver
```



O **pdfplumber** é ótimo para extrair texto, mas quando se trata de tabelas, ele tem limitações que precisamos considerar. Ele funciona razoavelmente bem com tabelas simples, mas não é a melhor opção quando:

- As células da tabela **não têm linhas visíveis** (sem bordas)
- A tabela está **mal alinhada visualmente**
- Há **mescla de células** (rowspan ou colspan)
- Tabelas com colunas muito próximas ou desalinhadas
- O PDF está em **duas colunas** e quebra o layout da tabela

EXPLORAR TEXTO EXTRAÍDO

Após extrair o texto completo, o próximo passo é **navegar pelo conteúdo** para localizar as partes que nos interessam.

Aqui usamos comandos simples de string para identificar palavras-chave e entender a estrutura do texto.

```
# Verifica se a palavra 'conclusão' aparece no texto
print("conclusão" in texto_completo.lower())

# Encontra a posição onde 'conclusão' aparece
indice = texto_completo.lower().find("conclusão")
print(f"Encontrado na posição: {indice}")

# Mostra os 500 caracteres após a palavra encontrada
print(texto_completo[indice:indice + 500])
```



Usamos `.lower()` para padronizar a busca (ignora maiúscula e minúscula)

EXTRAIR TABELAS

Em muitos documentos técnicos e regulatórios, as informações mais importantes não estão apenas no texto corrido, elas estão organizadas em tabelas. Essas tabelas concentram dados essenciais como listas de substâncias, limites, resultados de análises, cronogramas e decisões.

Nesta etapa, vamos ver como localizar, extrair e estruturar tabelas a partir de documentos em PDF, respeitando o layout e a organização visual do arquivo.

```
tabelas = []

with pdfplumber.open(caminho) as pdf:
    for pagina in pdf.pages:
        for tabela in pagina.extract_tables():
            if len(tabela) > 1:
                df = pd.DataFrame(tabela[1:], columns=tabela[0])
                tabelas.append(df)

# Visualiza a primeira tabela como exemplo
tabelas[0].head()
```



Para tabelas mais complexas, bibliotecas como camelot ou tabula-py oferecem suporte mais robusto, principalmente quando há muitas colunas, tabelas quebradas ou múltiplas páginas.

PyMuPDF (fitz)

O **PyMuPDF**, também conhecido como **fitz**, é uma biblioteca poderosa para trabalhar com PDFs em um nível mais detalhado.

Diferente do **pdfplumber**, que trabalha com o texto já processado, o **fitz** permite acessar o conteúdo com **posições exatas**, **estruturas visuais** e até **imagens** incorporadas.

Usamos o PyMuPDF porque ele nos dá:

- Mais controle sobre a **posição do texto na página** (coordenadas x, y)
- Funciona bem com PDFs com **colunas, blocos separados ou tabelas fora de linha**
- Permite **recortar áreas específicas** da página (crop)
- Suporte à extração de **imagens, metadados, links e até anotações**

```
pip install pymupdf  
import pymupdf
```

PyMuPDF (fitz)

O PyMuPDF permite extrair o conteúdo da página em formato de dicionário estruturado, com metadados sobre blocos de texto, coordenadas, estilo e ordem de leitura.

```
import pymupdf # pymupdf

# Abre o PDF
doc = pymupdf.open(caminho)

# Lê o texto da primeira página
pagina = doc[0]
texto = pagina.get_text()

print(texto[:1000]) # primeiros 1000 caracteres
```

A base do PyMuPDF é uma engine chamada **MuPDF** (escrita em C++) 

O módulo Python foi originalmente chamado de fitz, em homenagem ao executável fitz.exe da versão C++

O nome **“fitz”** foi mantido por compatibilidade, mesmo após a criação do pacote pymupdf

Portanto, pode ser que ao pesquisar sobre essa biblioteca ou usar ferramentas de LLM para escrever o código, podemos nos deparar com **import fitz**.

PyMuPDF (fitz)

Em muitos casos, a informação que precisamos está espalhada ao longo de várias páginas. Por isso, é importante saber como percorrer todas as páginas de um documento e extrair o conteúdo completo de forma automatizada.

Com o PyMuPDF, conseguimos fazer isso de maneira simples, usando um laço for para iterar página por página. O resultado é uma única string com todo o texto do documento, pronta para ser analisada ou processada.

```
import pymupdf

doc = pymupdf.open(caminho)

texto_completo = ""

for pagina in doc:
    texto_completo += pagina.get_text() + "\n"

print(texto_completo[:2000]) # primeiros 1000 caracteres
```


PyMuPDF (fitz)

Além disso, como ele oferece suporte para extração por posição. Podemos passar uma coordenada para remover elementos indesejados, como rodapé, margens ou cabeçalho

```
#Ler todas as páginas sem o rodapé
doc = pymupdf.open(caminho)

texto_completo = ""

# Região sem o rodapé: de y=25 até o topo (842)
regiao = Rect(0, 25, 595, 842)

for pagina in doc:
    texto_completo += pagina.get_textbox(regiao) + "\n"

print(texto_completo[:2000]) # mostra os primeiros 2000 caracteres
```

FERRAMENTAS PARA EXTRAÇÃO

Ferramenta	Melhor uso	Limitação
PDFPlumber	PDF textual simples	Layout complexo
PyMuPDF	Controle fino	Parsing manual
PyMuPDF4LLM	Markdown rápido para LLM	Não entende imagens
Marker	Parsing moderno e OCR	Mais pesado
Docling	Pipeline enterprise	Complexidade
Unstructured	RAG corporativo	Estratégias variam
MinerU	PDFs científicos complexos	Setup mais pesado

PyMuPDF4LLM

O **PyMuPDF4LLM** é uma extensão do PyMuPDF voltada para preparação de documentos para modelos de linguagem (LLMs).

Ele transforma PDFs em Markdown estruturado, preservando melhor:

- títulos
- seções
- listas
- ordem de leitura

```
import pymupdf4llm

markdown = pymupdf4llm.to_markdown("parecer.pdf")

print(markdown[:2000])
```



4LLM

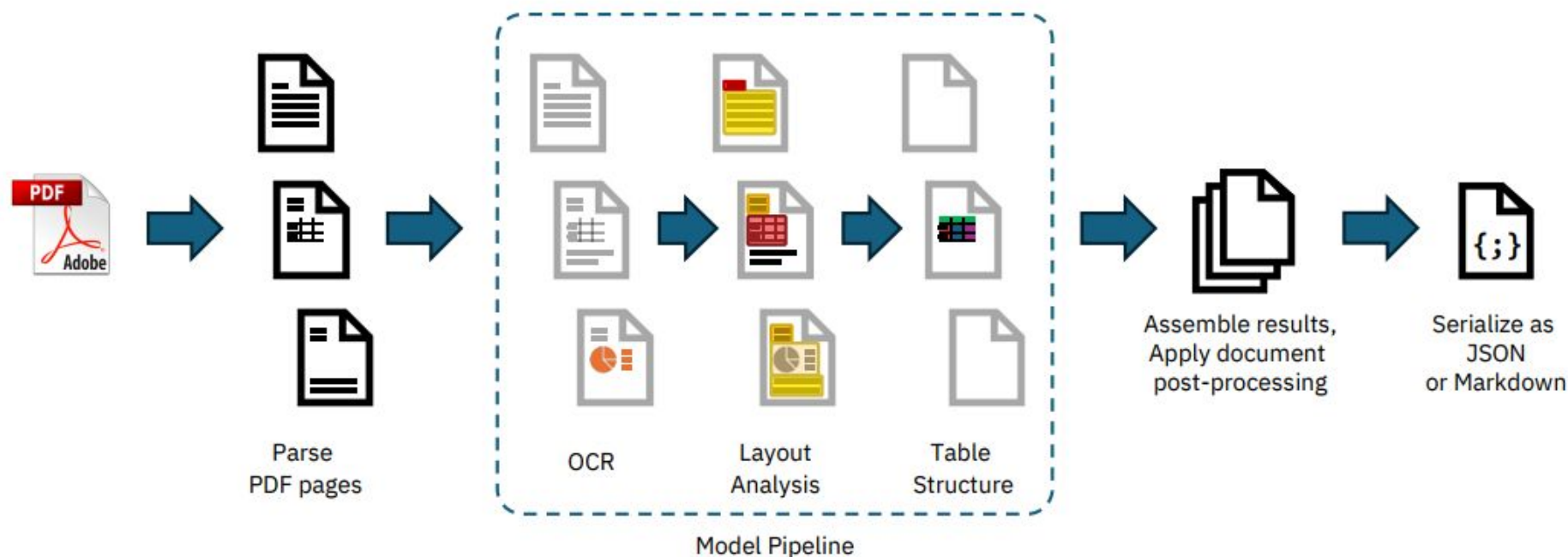
DOCLING

O **Docling** é uma ferramenta moderna que transforma documentos técnicos (PDF, DOCX, HTML, etc.) em **estruturas anotáveis e legíveis por IA**. Ele foi criado para facilitar a **organização semântica** de conteúdo, especialmente em contextos técnicos, regulatórios e científicos.



PRINCIPAIS RECURSOS DO DOCLING

- Suporte a diversos formatos: PDF, DOCX, HTML, imagens e planilhas
- Interpretação avançada de PDFs: mantém **layout, blocos, tabelas e leitura ordenada**
- Anotação semântica visual: permite marcar entidades, decisões, números técnicos, datas
- Exportação para formatos como JSON, HTML e Markdown
- Integrações com ferramentas como **LlamaIndex, LangChain, Haystack**
- Suporte a execução local (ideal para ambientes sensíveis ou offline)



EXTRAIR URL SIMPLES

Agora que já entendemos os principais elementos estruturais de uma página HTML, vamos explorar uma abordagem moderna para acessar e estruturar automaticamente o conteúdo de páginas públicas. O **Docling** permite ler documentos HTML diretamente a partir da URL, interpretar a organização do texto e gerar uma versão estruturada do conteúdo.

Vamos utilizar o **DocumentConverter**, componente principal da biblioteca, para acessar uma notícia publicada no site da Anvisa. O objetivo é transformar o conteúdo bruto da página em um formato limpo, organizado e pronto para análise, como Markdown.

```
from docling.document_converter import DocumentConverter

url =
"https://www.gov.br/anvisa/pt-br/assuntos/noticias-anvisa/2025/confira-os-destaques-da-6a-reunia
o-publica-da-dicol-de-2025"

converter = DocumentConverter()
result = converter.convert(url)

# Exporta o conteúdo estruturado em formato Markdown
markdown_result = result.document.export_to_markdown()

# Salva o resultado em um arquivo Markdown
print(markdown_result)
```



Você não precisa mais navegar por dezenas de <div> e <p> para encontrar o conteúdo certo. O Docling entende a estrutura da página e já entrega o texto limpo, como se alguém tivesse copiado e colado só o que importa.

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

LIMPEZA E PRÉ-PROCESSAMENTO

PREPARANDO O TEXTO EXTRAÍDO

Antes de estruturar ou analisar, é essencial fazer uma etapa de limpeza e padronização do conteúdo, pois, o texto extraído de um PDF raramente vem pronto para uso. Ele geralmente apresenta **ruídos estruturais** como:

- Quebras de linha fora de lugar
- Espaços duplicados
- Cabeçalhos repetidos em todas as páginas
- Palavras cortadas ou desalinhadas
- Resíduos de formatação visual que não fazem sentido no texto corrido

Ação	Objetivo	Função em Python
Remover <code>\n</code> soltos	Unir linhas que deveriam ser parágrafos	<code>str.replace("\n", " ")</code>
Remover espaços duplos	Corrigir quebras e layout	<code>str.replace(" ", " ")</code>
Padronizar caixa	Facilitar buscas e comparação	<code>.lower()</code> ou <code>.title()</code>
Corrigir acentuação (opcional)	Padronizar internacionalmente	<code>unidecode()</code>
Remover headers/repetições	Deixar texto mais limpo para análise	Regex ou regras manuais

PRÉ-PROCESSAMENTO DE DADOS

Após extrair o conteúdo do documento, o texto ainda contém **quebras artificiais, palavras fragmentadas e espaçamento irregular**, que dificulta qualquer análise posterior.

Este pré-processamento inicial tem como objetivo **unificar o texto**, corrigir pequenas distorções causadas pela extração do PDF e deixá-lo em um formato mais **limpo, contínuo e padronizado** e pronto para ser segmentado e estruturado.

```
import re

# Remove quebras de linha quebradas
texto_limpo = texto_completo.replace("\n", " ")

# Remove múltiplos espaços
texto_limpo = re.sub(r" {2,}", " ", texto_limpo)

# Remove palavras hifenizadas
texto_limpo = re.sub(r"-\\s+", "", texto_limpo)

# Remove espaços extras nas pontas
texto_limpo = texto_limpo.strip()

# Visualizar texto
print(texto_limpo[:2000])
```

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

ESTRUTURAÇÃO DA INFORMAÇÃO

PREPARANDO O TEXTO EXTRAÍDO

Depois da limpeza, o próximo passo é **organizar o conteúdo** do documento em partes significativas, como seções, campos ou registros. Essa estruturação permite que o texto bruto se torne um **conjunto de dados reutilizável**, seja em forma de dicionário, tabela ou JSON.

Em muitos documentos, precisamos **identificar padrões específicos** dentro de grandes blocos de texto, como:

- Números de processo
- Datas
- Números CAS
- Palavras-chave

Esses elementos não seguem exatamente a mesma frase, mas seguem um padrão de escrita.

As expressões regulares (ou **regex**) são uma ferramenta que permite **encontrar padrões textuais com precisão**.

EXPRESSÕES REGULARES

Vamos começar procurando um número de processo.

25351.903963/2022-21

Note que esse processo possui um padrão

XXXXX.XXXXXXX/XXXX-XX

```
# Expressão Regular para extrair processo
import re
re.findall(r"\d{5}\.\d{6}/\d{4}-\d{2}", texto_limpo)
```

- `\d{5}` → 5 dígitos
- `\.` → um ponto literal
- `\d{6}` → 6 dígitos
- `/` → barra
- `\d{4}` → 4 dígitos
- `-` → hífen
- `\d{2}` → 2 dígitos

EXPRESSÕES REGULARES

Expressão regular (ou **regex**) é uma linguagem utilizada para descrever padrões em textos. Com ela, conseguimos **identificar, extrair, validar** ou **substituir** partes de um texto que seguem uma estrutura específica, mesmo quando não sabemos exatamente o conteúdo.

É amplamente usada em programação para tarefas como:

- Encontrar datas, códigos, e-mails, números de documentos
- Validar formatos de entrada (como CPF ou CEP)
- Limpar ou transformar textos automaticamente

Para testar ou aprender mais sobre regex, acesse: regex101.com

Símbolo	Significado	Exemplo prático
\d	Um dígito (0 a 9)	\d\d\d\d\d\d\d → 12/03/2024
\w	Um caractere de palavra (letra, número, _)	\w+ → qualquer palavra
.	Qualquer caractere (exceto quebra de linha)	a.b → casa, a1b, axb
+	Um ou mais do elemento anterior	\d+ → 1, 123, 2024
*	Zero ou mais do elemento anterior	\w* → palavra ou vazio
{n}	Exatamente n repetições	\d{4} → 2024
{n,m}	De n a m repetições	\d{2,4} → 23, 2024
[...]	Qualquer um dos caracteres entre colchetes	[ABC] → A, B ou C
()	Agrupa uma parte do padrão	(\d{2}/\d{2}/\d{4}) → data completa
		OU lógico
^	Início da linha	^conclusão → linha que começa com "conclusão"
\$	Fim da linha	fim\$ → linha que termina com "fim"

EXTRAIR INFORMAÇÕES

```
# Expressões regulares
processos = re.findall(r"\d{5}\.\d{6}/\d{4}-\d{2}", texto_limpo)
datas = re.findall(r"\d{2}/\d{2}/\d{4}", texto_limpo)
cas_numbers = re.findall(r"\d{2,7}-\d{2}-\d", texto_limpo)

assinaturas = re.findall( r"documento assinado eletronicamente por (.+?),.*?em (\d{2}/\d{2}/\d{4})", texto_limpo, flags=re.IGNORECASE)

# Se houver assinatura, pega a primeira
nome_assinante = assinaturas[0][0] if assinaturas else None
data_assinatura = assinaturas[0][1] if assinaturas else None

# Organizar em DataFrame
dados_extraidos = pd.DataFrame([{
    "numero_processo": processos[0] if processos else None,
    "datas_mencionadas": ", ".join(datas),
    "numeros_cas": ", ".join(cas_numbers),
    "assinante": nome_assinante,
    "data_assinatura": data_assinatura
}])
```

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

ATIVIDADE PRÁTICA



MINISTÉRIO DA
SAÚDE



EXERCÍCIOS

Você está trabalhando com o conteúdo de mensagens institucionais extraídas de páginas públicas da Anvisa. Entre os textos extraídos, existem **informações úteis misturadas**, como telefones, datas e e-mails de contato.

Vamos utilizar expressões regulares (regex) para extrair todos os **endereços de e-mail válidos**, **número de telefone** e **Data da Reunião**.

```
texto = ""  
Entre em contato com nossa equipe:  
- ana.silva@anvisa.gov.br  
- suporte@empresa.com  
- regulatorio@anvisa.gov.br  
Telefone: (61) 99999-1234  
Data da reunião: 27/09/2024  
Envie dúvidas para: atendimento.anvisa@anvisa.gov.br ou contatar@empresa.org  
""
```

- Use a função `re.findall()`
- Considere que um e-mail válido contém uma sequência de caracteres, um @ e outra sequência (não precisa validar o domínio)
- Imprima a lista de e-mails encontrados
- Imprima a data da reunião
- Imprima o Telefone de Contato

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

REFERÊNCIAS



MINISTÉRIO DA
SAÚDE



REFERÊNCIAS

- **BeautifulSoup** – Documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc.ptbr/>
- **pdfplumber** – API Reference. <https://github.com/jsvine/pdfplumber>
- **PyMuPDF** – Documentation. <https://pymupdf.readthedocs.io/>
- **Docling** – GitHub Repository. <https://github.com/docling-project/docling>
- **REGEX101** – Regex Testing, Explanation and Reference. <https://regex101.com/>
- **Python – re** — Regular expression operations. <https://docs.python.org/3/library/re.html>
- **Requests** – HTTP for Humans. <https://docs.python-requests.org/>
- **Lutz, M. (2013)**. Learning Python (5th Edition). O'Reilly Media.
<https://www.oreilly.com/library/view/learning-python-5th/9781449355722/>
- **VanderPlas, J. (2016)**. Python Data Science Handbook. O'Reilly Media. <https://jakevdp.github.io/PythonDataScienceHandbook/>
- **McKinney, W. (2022)**. Python for Data Analysis. O'Reilly Media. <https://wesmckinney.com/book/>



CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL



MINISTÉRIO DA
SAÚDE

